SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: ITA 0607 **COURSE NAME: MACHINE LEARNING**

COURSE OUTCOME COVERED
CO2: *Neural Network Representation, Perceptron Model, Multilayer Perceptrons,*

Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)

Assessment Tool Weightage: 20%

QUESTIONS: 15 **Total Marks:25**

Annexure A

SCENARIO-BASED QUESTIONS

Code of Conduct:
I T . KODANDA RAM (192524162) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Annexure A

Short Answer Test

1. With the help of a neat sketch, explain the different functional blocks present in a machine learning system, including data collection, preprocessing, feature extraction, model training, and evaluation. Illustrate the working of the system by providing suitable examples for both supervised and unsupervised learning approaches, clearly highlighting how labelled and unlabelled data are handled within the learning process. (5 Marks)

2. Design a machine learning system for disaster management to predict events such as floods and earthquakes using both historical and real-time data sources. Explain the selection of an appropriate model, including whether a neural network or any other algorithm is suitable for this task, along with justification. Describe the data preprocessing steps and feature selection techniques required to improve model performance. Further, explain the complete process of training, validating, and evaluating the model, and discuss how such a system can assist authorities in early warning, risk assessment, and effective emergency response. (5 Marks)

3. In the context of neural networks, consider a multilayer perceptron used for binary classification. Explain how the backpropagation algorithm updates the weights in the network during training. Illustrate the process using a simple example by describing the forward pass, error calculation, and backward propagation of errors. Discuss how learning rate and activation functions influence the convergence and accuracy of the model. (5 Marks)

4. In the context of Genetic Algorithms, consider a dataset representing different candidate solutions for maximizing a fitness function based on two parameters: Speed and Accuracy. Each solution is evaluated as either suitable (Yes) or not suitable (No). Using the Genetic Algorithm approach, analyze how the population evolves by applying selection, crossover, and mutation operations. Show the step-by-step improvement of candidate solutions across generations and identify the best optimal solution. (5 Marks)

Dataset:

Example	Speed	Accuracy	Fitness Value	Suitable Solution
1	High	Low	40	No
2	Medium	Medium	60	Yes
3	High	Medium	75	Yes
4	Low	High	65	Yes
5	Medium	High	80	Yes

Explain how selection chooses the best individuals, how crossover generates new offspring, and how mutation helps in exploring the hypothesis space. Also, provide the final optimized solution after evolution.

5. In the context of Neural Networks for classification, consider a dataset used to predict whether a patient has a disease based on attributes such as Age, Blood Pressure, Glucose Level, and Lifestyle Risk. The output indicates whether the disease is present (Yes) or not (No). Using this dataset, explain how a neural network model can be trained using backpropagation to classify the data. Also, evaluate the model performance by comparing actual and predicted outputs. (5 Marks)

Dataset:

Example	Age	Blood Pressure	Glucose Level	Lifestyle Risk	Actual Output	Predicted Output
1	45	High	High	High	Yes	Yes
2	50	Normal	High	Medium	Yes	No
3	30	Normal	Normal	Low	No	No
4	60	High	High	High	Yes	Yes
5	40	Normal	Normal	Medium	No	Yes

Using the above dataset, compute the values of True Positives, True Negatives, False Positives, and False Negatives. Based on these values, analyze the performance of the neural network using suitable evaluation metrics. Also, explain how preprocessing and feature selection improve the model's accuracy and reliability.

6. With the help of a neat diagram, explain the architecture of a decision tree learning system. Describe how features are selected using measures such as entropy and information gain. Illustrate the process of tree construction and pruning using a small example dataset, and explain how overfitting can be controlled in decision tree models. (5 Marks)

7. Design a machine learning system for smart traffic management to predict traffic congestion using data such as vehicle density, time of day, weather conditions, and road type. Justify the choice of algorithm, explain preprocessing and feature engineering steps, and describe how the model can be trained, validated, and deployed for real-time decision-making. (5 Marks)

8. In the context of **Convolutional Neural Networks (CNNs)**, explain the different layers involved, including convolutional, pooling, and fully connected layers. Describe how feature maps are generated and how backpropagation updates weights in CNNs. Illustrate the process with an example of image classification and discuss factors affecting model performance. (5 Marks)

9. Consider the application of **Genetic Algorithms for feature selection** in a machine learning problem. Explain how candidate feature subsets are represented as chromosomes and how fitness is evaluated. Describe the role of selection, crossover, and mutation in evolving better feature subsets, and discuss how this improves model accuracy and reduces complexity. (5 Marks)

10. In the context of **classification problems**, explain how different evaluation metrics such as accuracy, precision, recall, F1-score, and ROC curve are used to assess model performance. Using a sample confusion matrix, demonstrate the

calculation of these metrics and discuss their importance in applications such as medical diagnosis and fraud detection. (5 Marks)

11. With the help of a neat diagram, explain the architecture of an **Artificial Neural Network (ANN)**. Describe the roles of input, hidden, and output layers, and explain how weights and biases are used in computation. Illustrate how forward propagation works with a simple example, and discuss how overfitting can be controlled using techniques such as dropout and regularization. (5 Marks)

12. Design a machine learning system for **weather forecasting** to predict rainfall using features such as temperature, humidity, wind speed, and atmospheric pressure. Justify the choice of algorithm, explain preprocessing and feature engineering steps, and describe how the model can be trained, validated, and deployed for real-time predictions. (5 Marks)

13. In the context of **Recurrent Neural Networks (RNNs)**, explain the architecture and how they handle sequential data. Describe the concept of hidden states and how information is passed through time steps. Illustrate with an example such as text prediction or time-series forecasting, and discuss challenges like vanishing gradients and possible solutions. (5 Marks)

14. Consider the use of **Genetic Algorithms for optimization problems** such as scheduling tasks in a system. Explain how solutions are encoded as chromosomes and how the fitness function is defined. Describe the processes of selection, crossover, and mutation, and discuss how these operations help in finding near-optimal solutions efficiently. (5 Marks)

15. In the context of **classification models**, explain how performance evaluation is carried out using metrics such as accuracy, precision, recall, F1-score, and ROC-AUC. Using a sample confusion matrix, demonstrate how each metric is calculated and discuss their importance in real-world applications such as spam detection and disease diagnosis. (5 Marks)

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**20**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**15**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Question 1. Functional Blocks of a Machine Learning System

Question as given:
With the help of a neat sketch, explain the different functional blocks present in a machine learning system, including data collection, preprocessing, feature extraction, model training, and evaluation. Illustrate the working of the system by providing suitable examples for both supervised and unsupervised learning approaches, clearly highlighting how labelled and unlabelled data are handled within the learning process. (5 Marks)

1. Architecture diagram

```
Data Collection (sensors, databases, files)
↓
Data Preprocessing (cleaning, encoding, scaling)
↓
Feature Extraction / Selection
↓
Model Training (labelled or unlabelled data)
↓
Model Evaluation
↓
Prediction / Deployment
```

2. Explanation of each block

- Data collection: Gather information from databases, sensors, surveys, files, or images.

- Data preprocessing: Handle missing values, remove duplicates, encode categories, and scale numerical features when needed.

- Feature extraction / selection: Identify useful characteristics and remove irrelevant or redundant inputs.

- Model training: Apply an algorithm to learn patterns and relationships from the training data.

- Model evaluation: Use separate validation/test data and suitable metrics to measure performance and errors.

- Prediction/deployment: Use the trained model on new data and monitor performance after deployment.

3. Supervised and unsupervised learning

Supervised learning	Unsupervised learning
Uses labelled data	Uses unlabelled data
Input includes a known target output	Target output is not supplied
Example: spam detection	Example: customer clustering
Algorithms: decision tree, neural network	Algorithms: K-means, hierarchical clustering

Example: In spam detection, emails are labelled as spam or not spam, and the model learns from these labels. In customer clustering, the algorithm groups customers by similarity without predefined group labels.

Conclusion / Final answer: The functional blocks convert raw data into predictions or discovered patterns through systematic data preparation, learning, and evaluation.

Question 2. Machine Learning System for Disaster Management

Question as given:
Design a machine learning system for disaster management to predict events such as floods and earthquakes using both historical and real-time data sources. Explain the selection of an appropriate model, including whether a neural network or any other algorithm is suitable for this task, along with justification. Describe the data preprocessing steps and feature selection techniques required to improve model performance. Further, explain the complete process of training, validating, and evaluating the model, and discuss how such a system can assist authorities in early warning, risk assessment, and effective emergency response. (5 Marks)

1. Proposed architecture

```
Historical records + real-time sensor/weather feeds
↓
Data integration and preprocessing
↓
Feature engineering and selection
↓
Train → validate → test model
↓
Event probability / risk score
↓
Warning dashboard for authorities
```

2. Data sources and model selection

Historical data may include rainfall, river levels, soil moisture, seismic readings, past flood locations, earthquake records, and damage reports. Real-time sources include rain gauges, river sensors, weather services, and seismic monitoring systems.

For flood risk using structured data, Random Forest or gradient-boosted trees are suitable baselines. LSTM or another time-series neural network may help when long sequences are available. Seismic event detection is a different task and may use signal processing with specialised neural networks. Earthquakes cannot be predicted with certainty; seismic early-warning systems generally detect an event after it starts and may provide limited warning.

3. Preprocessing and feature selection

- Synchronize timestamps and locations across data sources.

- Remove duplicates and invalid sensor readings; handle missing values carefully.

- Encode categories and scale numerical values when required by the algorithm.

- Create features such as rainfall totals, river-level rise rate, soil saturation, recent seismic signal statistics, and lagged readings.

- Select useful features using domain knowledge, feature importance, mutual information, or validation-based methods. Prevent future-data leakage.

4. Training, validation, and evaluation

Train on earlier events or locations, tune on validation data, and evaluate on held-out events or locations. For rare disasters, report precision, recall, F1-score, PR-AUC, false-alarm rate, and missed-event rate rather than accuracy alone. Test warning lead time and calibrate predicted probabilities.

5. Use in disaster response

The deployed system processes new observations and displays the location, risk level, confidence, and time of prediction. Authorities can use this information to prioritize inspections, prepare evacuation resources, and issue warnings under official procedures. Sensor failures, uncertainty, and model drift must be monitored.

Conclusion / Final answer: A validated ML system can support early warning and risk assessment, but it should complement official monitoring and expert judgement.

Question 3. Backpropagation in a Multilayer Perceptron

Question as given:
In the context of neural networks, consider a multilayer perceptron used for binary classification. Explain how the backpropagation algorithm updates the weights in the network during training. Illustrate the process using a simple example by describing the forward pass, error calculation, and backward propagation of errors. Discuss how learning rate and activation functions influence the convergence and accuracy of the model. (5 Marks)

1. Training cycle

```
Inputs → Forward pass → Predicted output
↓
Target → Calculate loss / error
↓
Backpropagate gradients through layers
↓
Update weights and biases
```

2. Forward pass — illustrative example

Assume a single sigmoid output neuron with x1 = 1, x2 = 0, w1 = 0.5, w2 = 0.4, bias b = -0.2, and target t = 1. These are example values because the question does not specify numerical values.

Formula: z = w1x1 + w2x2 + b

Substitution: z = (0.5 × 1) + (0.4 × 0) - 0.2 = 0.3

Sigmoid: y = 1 / (1 + e^(-z)) = 1 / (1 + e^(-0.3)) ≈ 0.5744

3. Error and backward pass

Using squared error: E = 1/2(t - y)^2 = 1/2(1 - 0.5744)^2 ≈ 0.0906.

For a sigmoid neuron with squared error, δ = (y - t)y(1 - y) ≈ (0.5744 - 1)(0.5744)(0.4256) ≈ -0.1042.

Gradient descent update: w_new = w_old - ηδx. With illustrative learning rate η = 0.1, w1_new = 0.5 - 0.1(-0.1042)(1) ≈ 0.5104. Since x2 = 0, w2_new = 0.4. Bias update: b_new = b - ηδ ≈ -0.1896.

In a multilayer perceptron, the chain rule propagates error gradients backward from the output layer through each hidden layer, allowing every weight to be updated.

4. Effect of learning rate and activation

- A very high learning rate may overshoot the minimum and destabilize training; a very low rate makes learning slow.

- Sigmoid maps outputs to 0–1 but can suffer vanishing gradients. ReLU is common in hidden layers; sigmoid is common for binary output.

Conclusion / Final answer: Backpropagation calculates gradients of the loss and updates weights and biases to reduce prediction error.

Question 4. Genetic Algorithm for Speed and Accuracy

Question as given:
In the context of Genetic Algorithms, consider a dataset representing different candidate solutions for maximizing a fitness function based on two parameters: Speed and Accuracy. Each solution is evaluated as either suitable (Yes) or not suitable (No). Using the Genetic Algorithm approach, analyze how the population evolves by applying selection, crossover, and mutation operations. Show the step-by-step improvement of candidate solutions across generations and identify the best optimal solution. Explain how selection chooses the best individuals, how crossover generates new offspring, and how mutation helps in exploring the hypothesis space. Also, provide the final optimized solution after evolution. (5 Marks)

Dataset (as provided in the question)

Example	Speed	Accuracy	Fitness Value	Suitable Solution
1	High	Low	40	No
2	Medium	Medium	60	Yes
3	High	Medium	75	Yes
4	Low	High	65	Yes
5	Medium	High	80	Yes

1. Selection

Fitness is already provided. Higher fitness is preferred, so Example 5 (80) and Example 3 (75) are strong parent candidates. Example 1 (40, unsuitable) would generally have a lower chance of selection.

2. Crossover

A chromosome must encode the speed and accuracy settings. Since the question does not specify an encoding, the following binary crossover is only an illustration, not a calculation of the original candidates' fitness.

```
Parent 1: 11 | 01
Parent 2: 10 | 10
Crossover after bit 2
Offspring 1: 11 | 10 = 1110
Offspring 2: 10 | 01 = 1001
```

3. Mutation and generation-by-generation evolution

Mutation randomly changes a gene or bit, introducing diversity. The process is repeated over generations:

Stage	Operation	Effect
Generation 0	Evaluate five given candidates	Best listed fitness is 80 (Example 5)
Generation 1	Select fitter candidates and cross them	Create new combinations
Variation	Mutate one or more genes	Explore additional candidates
Next generation	Evaluate offspring using the same fitness function	Retain stronger candidates and repeat

The exact fitness of new offspring cannot be computed without the chromosome encoding and a rule mapping the offspring back to speed, accuracy, and fitness.

Conclusion / Final answer: Among the supplied candidates, Example 5 (Medium speed, High accuracy) is the best listed solution: fitness = 80 and Suitable = Yes. A global optimum cannot be guaranteed from the supplied table alone.

Question 5. Neural Network Classification for Disease Prediction

Question as given:
In the context of Neural Networks for classification, consider a dataset used to predict whether a patient has a disease based on attributes such as Age, Blood Pressure, Glucose Level, and Lifestyle Risk. The output indicates whether the disease is present (Yes) or not (No). Using this dataset, explain how a neural network model can be trained using backpropagation to classify the data. Also, evaluate the model performance by comparing actual and predicted outputs. Using the above dataset, compute the values of True Positives, True Negatives, False Positives, and False Negatives. Based on these values, analyze the performance of the neural network using suitable evaluation metrics. Also, explain how preprocessing and feature selection improve the model's accuracy and reliability. (5 Marks)

Dataset (as provided in the question)

Example	Age	Blood Pressure	Glucose Level	Lifestyle Risk	Actual Output	Predicted Output
1	45	High	High	High	Yes	Yes
2	50	Normal	High	Medium	Yes	No
3	30	Normal	Normal	Low	No	No
4	60	High	High	High	Yes	Yes
5	40	Normal	Normal	Medium	No	Yes

1. Training and preprocessing

Encode categorical variables such as blood pressure, glucose level, and lifestyle risk. Scale age if required. During training, the network performs a forward pass, compares the predicted probability with the known label using a loss function, and uses backpropagation to update weights. Train and validate using a larger representative dataset; the five supplied rows are used here to demonstrate evaluation.

2. Confusion matrix counts

	Predicted Yes	Predicted No
Actual Yes	TP = 2	FN = 1
Actual No	FP = 1	TN = 1

TP = 2 (Examples 1 and 4); TN = 1 (Example 3); FN = 1 (Example 2); FP = 1 (Example 5). Total = 2 + 1 + 1 + 1 = 5.

3. Evaluation metrics

Accuracy = (TP + TN) / Total × 100 = (2 + 1) / 5 × 100 = 60%.

Precision = TP / (TP + FP) × 100 = 2 / 3 × 100 = 66.67%.

Recall = TP / (TP + FN) × 100 = 2 / 3 × 100 = 66.67%.

F1-score = 2 × Precision × Recall / (Precision + Recall) = 66.67%.

4. Feature selection and reliability

Removing irrelevant or redundant features can reduce complexity and overfitting. Feature selection must be performed using training data only. Missing values, inconsistent category labels, and class imbalance should be handled carefully. Because this is a very small dataset, these metrics illustrate the method and are not sufficient to establish clinical reliability.

Conclusion / Final answer: TP = 2, TN = 1, FP = 1, FN = 1. Accuracy = 60%, precision = 66.67%, recall = 66.67%, and F1-score = 66.67%.

Question 6. Decision Tree Learning System

Question as given:
With the help of a neat diagram, explain the architecture of a decision tree learning system. Describe how features are selected using measures such as entropy and information gain. Illustrate the process of tree construction and pruning using a small example dataset, and explain how overfitting can be controlled in decision tree models. (5 Marks)

1. Architecture diagram

```
Training dataset
↓
Choose split using entropy / information gain
↓
Decision node → branches by feature values
↓
Repeat until stopping condition
↓
Leaf node gives predicted class
↓
Prune branches that do not generalize
```

2. Entropy and information gain

Entropy measures class impurity: $H(S) = -\Sigma\ p_i \log2(p_i)$. Information gain for feature A is $IG(S,A) = H(S) - \Sigma_v (|S_v|/|S|)H(S_v)$. A split with greater information gain reduces uncertainty more.

3. Example calculation

Suppose 10 training records contain 6 positive and 4 negative cases. Initial entropy is:

$H(S) = -(6/10)\log2(6/10) - (4/10)\log2(4/10) \approx 0.971$ bits.

To calculate information gain for a particular feature, the positive/negative counts in each branch are also needed. Those counts are not specified, so a numeric gain for a specific feature cannot be calculated from the question alone.

```
Illustrative tree:
[Glucose high?]
/ \
Yes No
| |
[High risk] [Low risk]
| |
Disease No disease
```

4. Construction and pruning

- Start with all training samples at the root.

- Calculate candidate split scores and choose a useful feature.

- Repeat recursively for each branch until a stopping condition is met.

- Prune weak branches using validation data, maximum depth, minimum samples per leaf, or cost-complexity pruning.

Conclusion / Final answer: Entropy and information gain help select splits. Pruning and validation control overfitting and improve generalization.

Question 7. Machine Learning for Smart Traffic Management

Question as given:
Design a machine learning system for smart traffic management to predict traffic congestion using data such as vehicle density, time of day, weather conditions, and road type. Justify the choice of algorithm, explain preprocessing and feature engineering steps, and describe how the model can be trained, validated, and deployed for real-time decision-making. (5 Marks)

1. Architecture

```
Cameras / road sensors / weather feeds
↓
Data cleaning and feature engineering
↓
Train and validate congestion model
↓
Predict low / moderate / high congestion
↓
Dashboard for real-time traffic decisions
```

2. Algorithm selection

Random Forest or gradient-boosted trees are practical starting choices for mixed numerical and categorical data. If long traffic sequences are available, an LSTM or other time-series model can also be tested. Select the model based on validation quality, inference speed, and interpretability.

3. Preprocessing and feature engineering

- Inputs: vehicle count/density, average speed, time, day, weather, road type, incidents, and recent congestion.

- Remove duplicate readings, flag faulty sensors, and handle missing values.

- Encode road type and weather; scale numerical inputs when needed.

- Create features such as vehicles per minute, change in average speed, rush-hour indicator, and rolling averages.

- Define congestion labels consistently, for example using speed relative to free-flow speed.

4. Training, validation, and deployment

Use time-based splits so future data do not leak into training. Evaluate using a confusion matrix, per-class precision/recall/F1, and prediction latency. Test across roads and peak/off-peak periods. In deployment, new readings are processed and the dashboard shows congestion class, location, and confidence. Operators can adjust signals or suggest routes, with human oversight and fallback rules.

Conclusion / Final answer: A real-time traffic model supports better decisions when trained on representative data, validated over time, and monitored after deployment.

Question 8. Convolutional Neural Networks (CNNs)

Question as given:
In the context of Convolutional Neural Networks (CNNs), explain the different layers involved, including convolutional, pooling, and fully connected layers. Describe how feature maps are generated and how backpropagation updates weights in CNNs. Illustrate the process with an example of image classification and discuss factors affecting model performance. (5 Marks)

1. CNN architecture

```
Input image
↓
Convolution + activation → feature maps
↓
Pooling → smaller feature maps
↓
Repeated convolution / pooling
↓
Flatten or global pooling
↓
Fully connected layer → class probabilities
```

2. Layers and feature maps

- Convolution: Learnable filters slide over local image regions and detect patterns such as edges and textures.

- Activation: ReLU introduces non-linearity by replacing negative values with zero.

- Pooling: Max or average pooling reduces spatial dimensions and computation.

- Fully connected/output layer: Combines features to predict classes, often with softmax for multiple classes.

3. Example convolution calculation

Use the input patch I and filter K below. The filter is applied with stride 1 and no padding.

Input patch I	Filter K
1 2 0	1 0
0 1 3	0 -1
2 1 0	

Top-left: (1×1)+(2×0)+(0×0)+(1×−1)=0.

Top-right: (2×1)+(0×0)+(1×0)+(3×−1)=−1.

Bottom-left: (0×1)+(1×0)+(2×0)+(1×−1)=−1.

Bottom-right: (1×1)+(3×0)+(1×0)+(0×−1)=1.

Output	Feature map
Before ReLU	0 -1; -1 1
After ReLU	0 0; 0 1

4. Backpropagation and performance

The loss compares predicted probabilities with the correct image label. Backpropagation calculates gradients for convolutional filters and dense-layer weights; an optimizer such as SGD or Adam updates these weights.

Performance depends on data quality and size, image resolution, network depth, learning rate, batch size, class balance, regularization, and data augmentation. Validation and test data must remain separate from training data.

Conclusion / Final answer: CNNs learn spatial features through convolution, reduce dimensions through pooling, and use the learned features to classify images.

Question 9. Genetic Algorithm for Feature Selection

Question as given:
Consider the application of Genetic Algorithms for feature selection in a machine learning problem. Explain how candidate feature subsets are represented as chromosomes and how fitness is evaluated. Describe the role of selection, crossover, and mutation in evolving better feature subsets, and discuss how this improves model accuracy and reduces complexity. (5 Marks)

1. Chromosome representation

For five candidate features, a binary chromosome indicates whether each feature is selected (1) or excluded (0).

Chromosome	Selected features
11010	Features 1, 2, and 4
00111	Features 3, 4, and 5
11111	All five features

2. Fitness function

An illustrative fitness function is: Fitness = validation score – λ × (number of selected features / total features). Here λ controls the penalty for complexity. The validation score may be F1-score or accuracy, depending on the task.

3. Selection, crossover, and mutation

- Selection: Higher-fitness feature subsets have a greater chance of becoming parents.

- Crossover: Combine sections of parent chromosomes. For example, 11|010 and 00|111 may produce 11111 and 00010.

- Mutation: Flip a bit, for example 11010 → 10010, to explore another subset.

4. Evolution cycle and benefit

```
Generate candidate subsets
↓
Evaluate each subset on validation data
↓
Select higher-fitness candidates
↓
Crossover and mutation
↓
Repeat until stopping condition
↓
Return best validated subset
```

Feature selection can reduce computation, improve interpretability, and sometimes reduce overfitting. It does not guarantee higher accuracy. Perform selection only within the training/validation process to avoid test-set leakage.

Conclusion / Final answer: The best subset balances predictive performance with a small, useful set of features, as measured by the defined fitness function.

Question 10. Classification Evaluation Metrics and ROC Curve

Question as given:
In the context of classification problems, explain how different evaluation metrics such as accuracy, precision, recall, F1-score, and ROC curve are used to assess model performance. Using a sample confusion matrix, demonstrate the calculation of these metrics and discuss their importance in applications such as medical diagnosis and fraud detection. (5 Marks)

1. Sample confusion matrix

	Predicted Positive	Predicted Negative	Total actual
Actual Positive	TP = 40	FN = 5	45
Actual Negative	FP = 10	TN = 50	60
Total predicted	50	55	105

Total = TP + TN + FP + FN = 40 + 50 + 10 + 5 = 105.

2. Step-by-step calculations

Accuracy = (TP + TN) / Total × 100 = (40 + 50) / 105 × 100 = 85.71%.

Precision = TP / (TP + FP) × 100 = 40 / 50 × 100 = 80.00%.

Recall = TP / (TP + FN) × 100 = 40 / 45 × 100 = 88.89%.

F1-score = 2TP / (2TP + FP + FN) × 100 = 80 / 95 × 100 = 84.21%.

Specificity = TN / (TN + FP) × 100 = 50 / 60 × 100 = 83.33%.

3. ROC curve

False-positive rate = FP / (FP + TN) = 10 / 60 = 0.1667 (16.67%). True-positive rate = TP / (TP + FN) = 40 / 45 = 0.8889 (88.89%).

A ROC curve plots true-positive rate against false-positive rate at different thresholds. ROC-AUC is the area under the full curve and cannot be determined from one confusion matrix at a single threshold.

4. Importance

Accuracy gives overall correctness. Precision matters when false alarms are costly; recall matters when missing positive cases is costly. F1 balances precision and recall. In medical diagnosis and fraud detection, the cost of false positives and false negatives should guide metric selection.

Conclusion / Final answer: Accuracy = 85.71%, precision = 80.00%, recall = 88.89%, F1-score = 84.21%, FPR = 16.67%, and TPR = 88.89%. ROC-AUC requires predictions at multiple thresholds.

Question 11. Artificial Neural Network Architecture and Forward Propagation

Question as given:
With the help of a neat diagram, explain the architecture of an Artificial Neural Network (ANN). Describe the roles of input, hidden, and output layers, and explain how weights and biases are used in computation. Illustrate how forward propagation works with a simple example, and discuss how overfitting can be controlled using techniques such as dropout and regularization. (5 Marks)

1. Architecture diagram



2. Roles of layers, weights, and biases

The input layer receives feature values. Hidden layers compute weighted sums and apply activations to learn patterns. The output layer returns a value or class probability. Weights control connection strength; biases shift the weighted sum.

3. Forward propagation — illustrative calculation

Assume x1 = 1, x2 = 2, w1 = 0.5, w2 = -0.25, and b = 0.1.

Formula: z = w1x1 + w2x2 + b

Substitution: z = (0.5×1) + (-0.25×2) + 0.1 = 0.5 - 0.5 + 0.1 = 0.1.

Sigmoid output: y = 1/(1 + e^(-0.1)) ≈ 0.5250. With a threshold of 0.5, the illustrative predicted class is 1.

4. Overfitting control

- Dropout randomly disables selected neurons during training.

- L1/L2 regularization penalizes overly large weights.

- Early stopping monitors validation performance and stops when it no longer improves.

- Representative data, augmentation where suitable, and a simpler architecture can improve generalization.

Conclusion / Final answer: ANNs transform inputs using weighted connections, biases, and activation functions. Regularization and validation help control overfitting.

Question 12. Machine Learning System for Weather Forecasting

Question as given:
Design a machine learning system for weather forecasting to predict rainfall using features such as temperature, humidity, wind speed, and atmospheric pressure. Justify the choice of algorithm, explain preprocessing and feature engineering steps, and describe how the model can be trained, validated, and deployed for real-time predictions. (5 Marks)

1. Architecture

```
Weather stations / forecast feeds
↓
Clean and synchronize observations
↓
Create features and lag variables
↓
Train → validate → test rainfall model
↓
Rainfall probability or amount
↓
Real-time forecast dashboard
```

2. Algorithm choice

Random Forest or gradient-boosted trees are suitable baselines for tabular weather data. LSTM or another time-series model may help when temporal sequences are important. Regression predicts rainfall amount; classification predicts rain/no rain or whether rainfall exceeds a chosen threshold.

3. Preprocessing and feature engineering

- Collect temperature, humidity, wind speed, pressure, timestamp, location, and historical rainfall.
- Remove duplicates, check units, flag sensor errors, and handle missing values.
- Create lagged values, rolling averages, pressure changes, and humidity trends.
- Define the target clearly, such as rainfall amount in the next six hours or rain/no rain.
- Scale/encode inputs as required by the selected algorithm.

4. Training, evaluation, and deployment

Use chronological splits: earlier periods for training, a later period for validation, and a final unseen period for testing. For rainfall amount, use MAE or RMSE. For rain/no-rain classification, use precision, recall, F1-score, PR-AUC, and calibration. Compare with a simple baseline.

At deployment, process the latest observations with the same feature pipeline used during training. Return a timestamped forecast and confidence estimate. Monitor errors by season and location and retrain if performance drifts.

Conclusion / Final answer: A useful rainfall prediction system combines reliable observations, time-aware validation, a clearly defined target, and continuous monitoring.

Question 13. Recurrent Neural Networks and Sequential Data

Question as given:
In the context of Recurrent Neural Networks (RNNs), explain the architecture and how they handle sequential data. Describe the concept of hidden states and how information is passed through time steps. Illustrate with an example such as text prediction or time-series forecasting, and discuss challenges like vanishing gradients and possible solutions. (5 Marks)

1. RNN architecture

```
Input x1 → [RNN cell] → hidden state h1
↓
Input x2 → [RNN cell] → hidden state h2
↓
Input x3 → [RNN cell] → hidden state h3 → output
```

2. Hidden state

An RNN processes a sequence one step at a time and carries information in a hidden state.

Equations: h_t = tanh(W_x x_t + W_h h_(t-1) + b_h); y_t = g(W_y h_t + b_y). The current input and previous hidden state jointly determine the new hidden state.

3. Illustrative example

If the combined input to tanh at one time step is 1.2, then h_t = tanh(1.2) ≈ 0.8337. The next step uses this state with the next input. For text prediction, the sequence “the sky is” can help predict the next word; for forecasting, past temperatures or traffic levels can predict a future value.

4. Training challenges and solutions

RNNs are trained using backpropagation through time, which propagates gradients through sequence steps.

- Vanishing gradients make long-range dependencies hard to learn.

- Exploding gradients can destabilize training; gradient clipping helps.

- LSTM and GRU cells, suitable initialization, normalization, and careful sequence length can improve learning.

Conclusion / Final answer: RNN hidden states pass information through time and support sequential tasks, but basic RNNs can struggle with long-range dependencies.

Question 14. Genetic Algorithm for Task Scheduling

Question as given:
Consider the use of Genetic Algorithms for optimization problems such as scheduling tasks in a system. Explain how solutions are encoded as chromosomes and how the fitness function is defined. Describe the processes of selection, crossover, and mutation, and discuss how these operations help in finding near-optimal solutions efficiently. (5 Marks)

1. Chromosome encoding

A chromosome represents one schedule. For four tasks A, B, C, and D on one processor, [B, A, D, C] is one possible execution order. For multiple processors, the encoding must also indicate processor assignments.

2. Fitness function with example

If the objective is to minimize makespan C, an illustrative fitness function is Fitness = 1/(1 + C), so smaller completion times give higher fitness.

Schedule A: C = 20, so Fitness = 1/(1+20) = 1/21 ≈ 0.0476.

Schedule B: C = 15, so Fitness = 1/(1+15) = 1/16 = 0.0625. Schedule B is preferred because it has a shorter makespan and higher fitness.

3. Selection, crossover, and mutation

- Selection: Give better schedules a higher chance of becoming parents.

- Crossover: Combine parent schedules using a permutation-preserving method such as order crossover so tasks are not duplicated.

- Mutation: Swap two tasks, move a task, or reassign a task to another processor.

4. Evolution

```
Generate valid schedules
↓
Calculate makespan and penalties
↓
Select parents
↓
Crossover + mutation
↓
Repair invalid schedules if required
↓
Keep best candidates and repeat
```

A good fitness function should also penalize deadline violations and resource conflicts. Genetic Algorithms can find near-optimal schedules efficiently but do not guarantee the global optimum.

Conclusion / Final answer: Genetic Algorithms search possible schedules by encoding them as chromosomes and repeatedly applying selection, crossover, and mutation.

Question 15. Classification Metrics and ROC-AUC

Question as given:
In the context of classification models, explain how performance evaluation is carried out using metrics such as accuracy, precision, recall, F1-score, and ROC-AUC. Using a sample confusion matrix, demonstrate how each metric is calculated and discuss their importance in real-world applications such as spam detection and disease diagnosis. (5 Marks)

1. Illustrative sample confusion matrix

	Predicted Positive / Spam	Predicted Negative / Not spam	Total actual
Actual Positive	TP = 30	FN = 5	35
Actual Negative	FP = 10	TN = 55	65
Total predicted	40	60	100

The numbers above are illustrative because the question does not provide a numerical confusion matrix.

2. Step-by-step calculations

Total = 30 + 55 + 10 + 5 = 100.

Accuracy = (TP + TN)/Total × 100 = (30+55)/100 × 100 = 85%.

Precision = TP/(TP+FP) × 100 = 30/(30+10) × 100 = 75%.

Recall = TP/(TP+FN) × 100 = 30/(30+5) × 100 ≈ 85.71%.

F1-score = 2TP/(2TP+FP+FN) × 100 = 60/(60+10+5) × 100 = 80%.

3. ROC-AUC

False-positive rate = FP/(FP+TN) = 10/65 ≈ 0.1538 (15.38%). True-positive rate = TP/(TP+FN) = 30/35 ≈ 0.8571 (85.71%).

The ROC curve plots TPR against FPR at multiple thresholds. ROC-AUC is the area under the complete curve; a single confusion matrix provides only one point and is not enough to calculate AUC.

4. Importance in real applications

In spam detection, precision matters because a false positive may hide a genuine email. In disease diagnosis, recall can be especially important because a false negative may miss a patient needing follow-up. Accuracy alone can mislead when classes are imbalanced.

Conclusion / Final answer: For the illustrative matrix: accuracy = 85%, precision = 75%, recall = 85.71%, F1-score = 80%, FPR = 15.38%, and TPR = 85.71%. AUC requires predictions at multiple thresholds.